Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки Кафедра
обчислювальної техніки

ЗВІТ

з лабораторної роботи №4

«Аналітичні SQL-запити (OLAP)»

з дисципліни: Бази Даних

Виконали: студенти групи ІМ-54

Дрига О.О.

Голубков В.С.

Ретізник А.Є.

Перевірив: Хмельницький А.А.

Київ - 2026

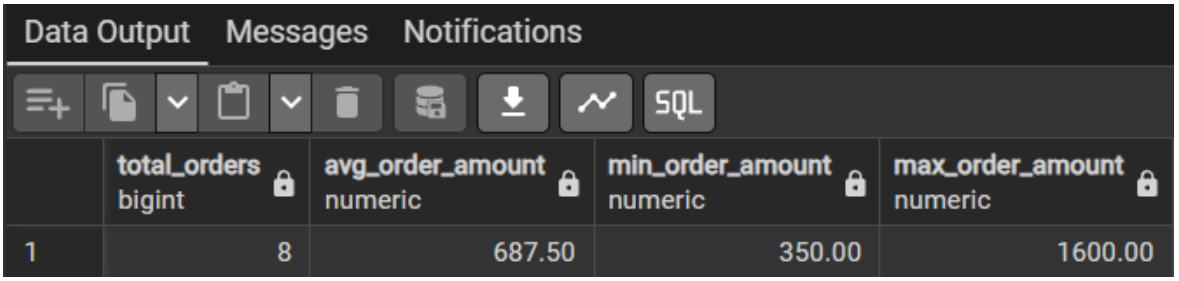
Тема: Аналітичні SQL-запити (OLAP)

Мета: навчитися писати аналітичні запити в PostgreSQL: обчислювати зведену статистику агрегатними функціями (COUNT, SUM, AVG, MIN, MAX), групувати дані через GROUP BY та фільтрувати групи через HAVING, об'єднувати кілька таблиць різними типами JOIN (INNER, LEFT, RIGHT, FULL, CROSS), використовувати підзапити в SELECT, WHERE та HAVING, а також перевіряти правильність результатів і пояснювати, що робить кожен запит.

```
select
count(order_id) as total_orders,
round(avg(total_amount), 2) as avg_order_amount,
min(total_amount) as min_order_amount,
max(total_amount) as max_order_amount
from orders;
```

Що робить: Запит застосовує агрегатні функції до таблиці orders. Він обчислює загальну кількість оформлених замовлень, середню вартість чека (округлену до двох знаків після коми за допомогою функції ROUND), а також знаходить найменшу та найбільшу суми замовлень.

Чому: Оскільки тут немає команди GROUP BY, база даних сприймає абсолютно всю таблицю orders як одну велику групу. Вона бере колонку total_amount, проходить по ній зверху донизу, збирає всі числа і видає один-єдиний рядок з результатами (одну суму, одне середнє, один максимум).



	total_orders bigint	avg_order_amount numeric	min_order_amount numeric	max_order_amount numeric
1	8	687.50	350.00	1600.00

```
select status,
count(order_id) as orders_count,
sum(total_amount) as total_revenue
from orders
group by status;
```

Що робить: Запит групує всі записи таблиці orders за колонкою status (статус замовлення). Для кожної утвореної групи вираховується кількість замовлень та сумарний виторг (SUM).

Чому: SQL бере всі замовлення і розкидає їх по групах (окремо 'Pending', 'Shipped' тощо). Тільки після цього він повертається до SELECT і застосовує COUNT та SUM окремо всередині кожної групи. Тому в результаті ми отримуємо не один рядок, а стільки рядків, скільки у нас унікальних статусів.

	status order_status	orders_count bigint	total_revenue numeric
1	Shipped	2	840.00
2	Processing	2	760.00
3	Delivered	2	3200.00
4	Pending	2	700.00

```
select b.title,
       b.isbn,
       b.price,
       p.name_publisher
from book b
inner join publisher p on b.publisher_id = p.publisher_id
where b.price > 350;
```

Що робить: Виконує внутрішнє з'єднання (INNER JOIN) таблиць book (книги) та publisher (видавництва) за спільним ключем publisher_id. Фільтрує результат за допомогою WHERE, залишаючи лише ті записи, де ціна книги перевищує 350 грн.

Чому: База даних шукає збіги за спільним полем publisher_id у таблицях book та publisher, щоб об'єднати їхні дані в один рядок. До результату потрапляють виключно ті записи, які існують в обох таблицях. Якщо у книги не вказано видавництво, вона ігнорується. З утвореного списку відбираються лише ті рядки, які відповідають умові — ціна книги більша за 350.

	title character varying (255)	isbn character varying (20)	price numeric (10,2)	name_publisher character varying (100)
1	Кобзар	234-DM	1300.00	Видавництво Старого Ле...
2	Інтернат	9789662909494	420.00	Видавництво Старого Ле...
3	Clean Code: A Handbook of Agile Software Craftsmans...	9780132350884	1450.00	O'Reilly Media
4	Гаррі Поттер і філософський камінь	9786171218567	380.00	Клуб Сімейного Дозвілля

```
select book_id, title, isbn, price
from book
where price > (select avg(price) from book);
```

Що робить: Запит використовує вкладений підзапит (SELECT AVG(price) FROM book) для динамічного обчислення середньої ціни всіх книг у базі. Головний запит виводить лише ті книги, ціна яких є більшою за це обчислене значення.

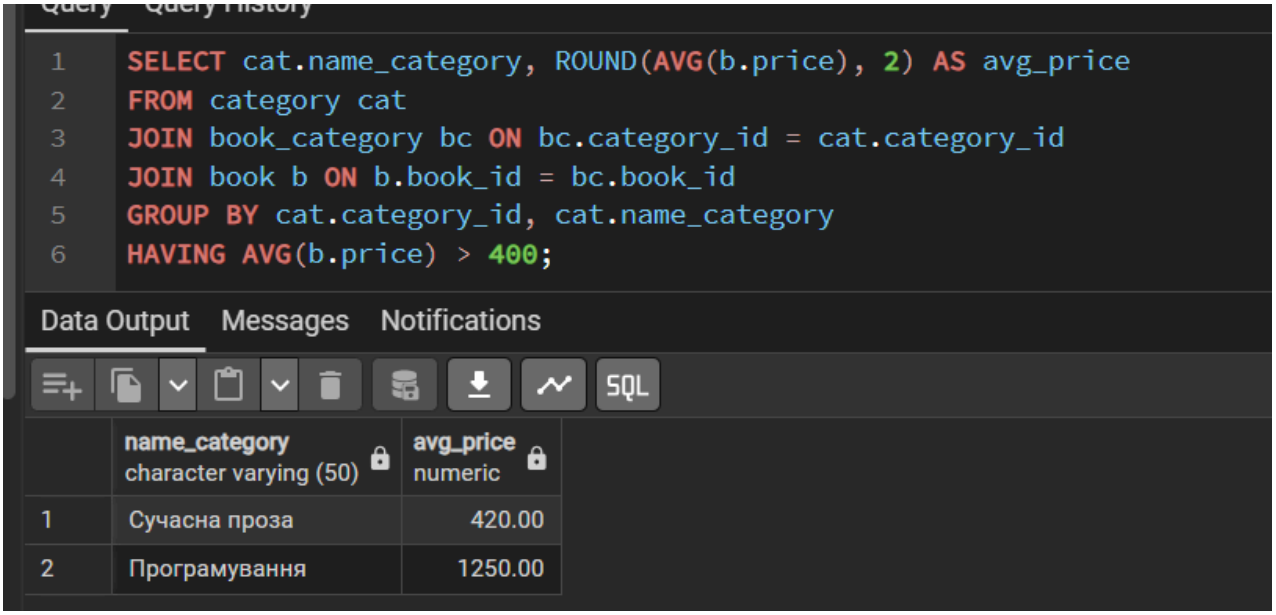
Чому: Спочатку SQL повністю ігнорує зовнішній запит і виконує лише те, що в дужках: SELECT AVG(price) FROM book. Цей внутрішній запит повертає одне конкретне число (наприклад, 420.50). SQL підставляє це число замість дужок. Тепер умова виглядає як WHERE price > 420.50. Після цього запускається головний запит, який просто йде по кожній книзі і порівнює її ціну з цим фіксованим числом.

	book_id [PK] integer	title character varying (255)	isbn character varying (20)	price numeric (10,2)
1	1	Clean Code: A Handbook of Agile Software Craftsmans...	9780132350884	1450.00
2	9	Кобзар	234-DM	1300.00

```
SELECT cat.name_category, ROUND(AVG(b.price), 2) AS avg_price
FROM category cat
JOIN book_category bc ON bc.category_id = cat.category_id
JOIN book b ON b.book_id = bc.book_id
GROUP BY cat.category_id, cat.name_category
HAVING AVG(b.price) > 400;
```

Що робить: Запит знаходить категорії книг, середня ціна книг у яких перевищує 400 грн, і виводить назву категорії та обчислену середню ціну.

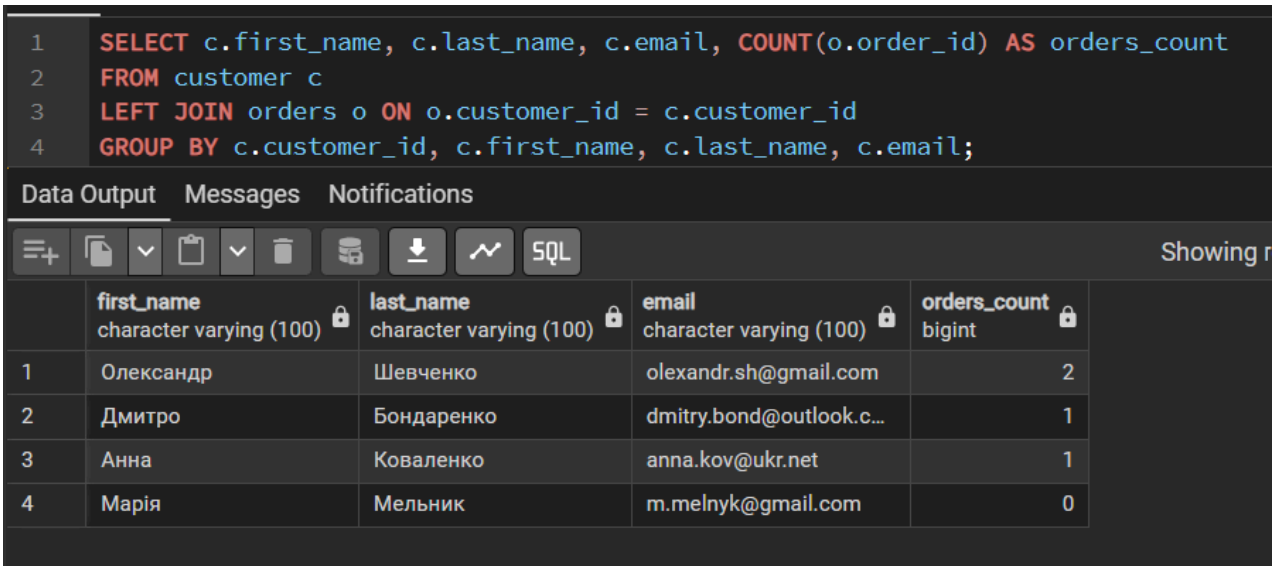
Чому: Книги та категорії пов'язані через проміжну таблицю book_category, тому база даних з'єднує три таблиці (category, book_category, book) в одну. Потім GROUP BY розбиває рядки на групи за категорією, і для кожної групи AVG рахує середню ціну. Лише після цього працює HAVING: він відкидає групи, де середня ціна не більша за 400. Умову AVG(b.price) > 400 не можна написати у WHERE, бо WHERE виконується до групування, коли середніх ще немає. У результаті залишаються «Сучасна проза» (420.00) та «Програмування» (1250.00).



```
SELECT c.first_name, c.last_name, c.email, COUNT(o.order_id) AS orders_count
FROM customer c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email;
```

Що робить: Запит виводить список усіх клієнтів (ім'я, прізвище, email) та кількість їхніх замовлень, зокрема й тих клієнтів, які ще не зробили жодного замовлення.

Чому: LEFT JOIN залишає всі рядки лівої таблиці customer, навіть якщо в таблиці orders для них немає пари. У таких випадках замість даних замовлення підставляється NULL. Функція COUNT(o.order_id) рахує лише значення, що не є NULL, тому клієнтка без замовлень (Мельник) отримує 0. Якби ми написали COUNT(*), вона отримала б помилкове значення 1, бо рядок із NULL теж був би порахований. GROUP BY групує рядки за клієнтом, щоб кількість замовлень рахувалась окремо для кожного.

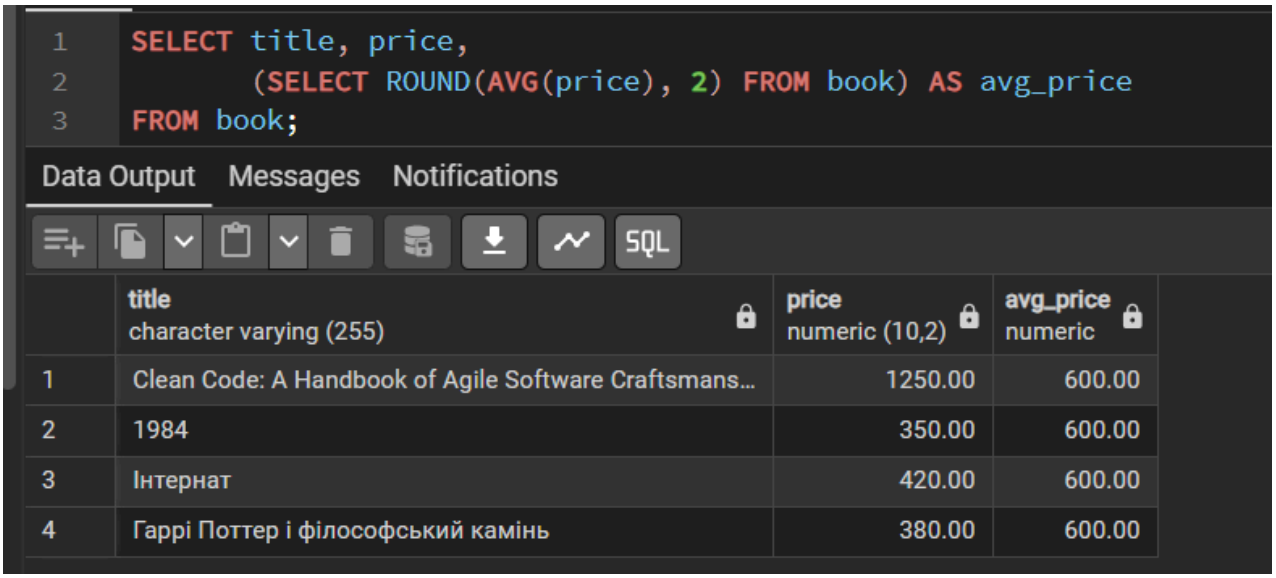


	first_name character varying (100)	last_name character varying (100)	email character varying (100)	orders_count bigint
1	Олександр	Шевченко	olexandr.sh@gmail.com	2
2	Дмитро	Бондаренко	dmitry.bond@outlook.c...	1
3	Анна	Коваленко	anna.kov@ukr.net	1
4	Марія	Мельник	m.melnyk@gmail.com	0

```
SELECT title, price,
       (SELECT ROUND(AVG(price), 2) FROM book) AS avg_price
FROM book;
```

Що робить: Запит виводить назву та ціну кожної книги і додає стовпець avg_price із загальною середньою ціною всіх книг магазину, яку обчислює підзапит у секції SELECT.

Чому: Підзапит (SELECT ROUND(AVG(price), 2) FROM book) не залежить від зовнішнього запиту, тому виконується один раз і повертає єдине число (тут 600.00). Це число підставляється в окремий стовпець avg_price у кожному рядку результату. Завдяки цьому ціну кожної книги можна одразу порівняти із середньою без додаткових запитів.

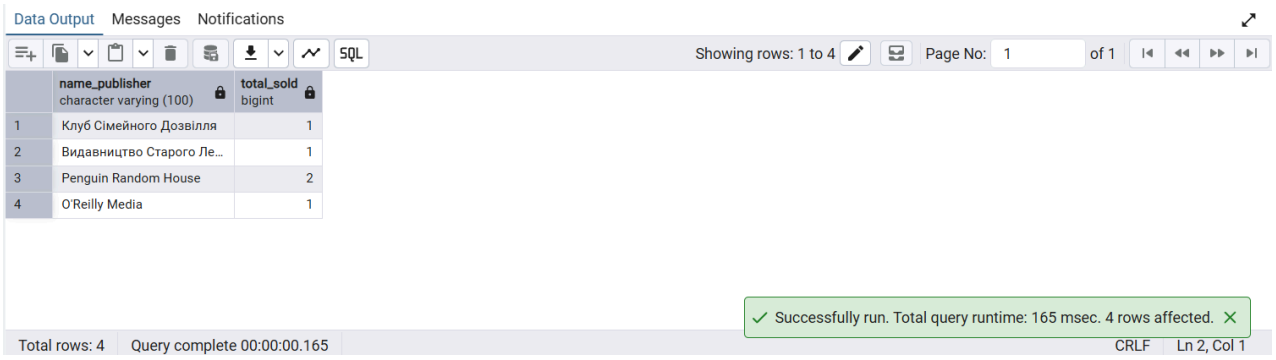


	title character varying (255)	price numeric (10,2)	avg_price numeric
1	Clean Code: A Handbook of Agile Software Craftsmans...	1250.00	600.00
2	1984	350.00	600.00
3	Інтернат	420.00	600.00
4	Гаррі Поттер і філософський камінь	380.00	600.00

```
SELECT publisher.name_publisher, SUM(order_item.quantity) AS total_sold
FROM publisher
JOIN book ON publisher.publisher_id = book.publisher_id
JOIN order_item ON book.book_id = order_item.book_id
GROUP BY publisher.name_publisher;
```

Що робить: Запит обчислює загальну кількість проданих примірників книг окремо для кожного видавництва.

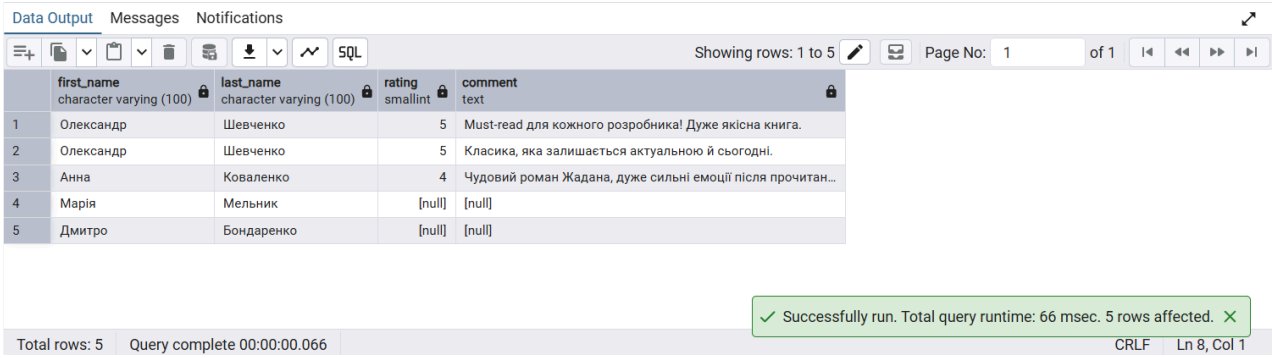
Чому: Оскільки видавництва та продані позиції не мають прямого зв'язку, використовуємо таблицю book як міст для їх об'єднання через JOIN. База даних з'єднує ці три таблиці в одну, розбиває рядки на групи за назвою видавництва, і всередині кожної групи додає кількість проданих штук.




```
SELECT customer.first_name, customer.last_name, review.rating, review.comment
FROM customer
FULL JOIN review ON customer.customer_id = review.customer_id;
```

Що робить: Виводить імена клієнтів разом з їхніми оцінками та коментарями, включаючи клієнтів без відгуків та можливі відгуки без клієнтів.

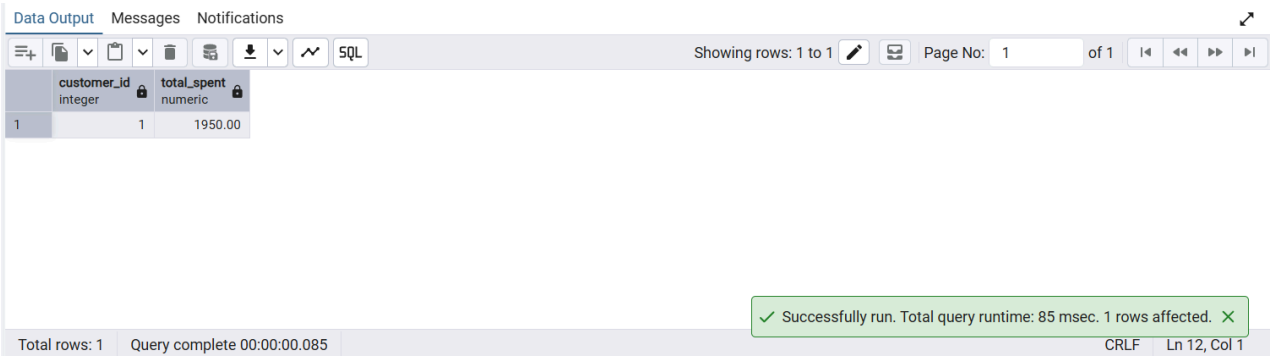
Чому: FULL JOIN примусово бере всі рядки з обох таблиць. Якщо у клієнта немає відгуку, база підставить у колонки rating та comment значення NULL.



```
SELECT temp.customer_id, temp.total_spent
FROM (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
) AS temp
WHERE temp.total_spent > 500;
```

Що робить: Знаходить сумарні витрати по кожному клієнту і виводить лише тих покупців, які загалом витратили більше 500 грн.

Чому: Спочатку база даних виконує підзапит у дужках. Він рахує суми для кожного клієнта і створює в пам'яті тимчасову віртуальну таблицю, яку назвали temp. Після цього запускається зовнішній SELECT, який працює з temp як зі звичайною таблицею і легко фільтрує її за допомогою WHERE temp.total_spent > 500 (оскільки агрегатні функції не можна напряму писати у WHERE).



Висновок: У ході лабораторної роботи №4 було засвоєно основні прийоми аналітичної обробки даних (OLAP) у PostgreSQL на прикладі бази даних книжкового магазину. Практично написано запити з агрегатними функціями та групуванням (зведена статистика замовлень, виручка за статусами, видавництвами та клієнтами), а також запити з JOIN різних типів і з підзапитами в WHERE, SELECT та HAVING. Правильність кожного запиту перевірено порівнянням результату з даними в таблицях. З'ясовано, що WHERE фільтрує окремі рядки до групування, а HAVING фільтрує групи після обчислення агрегатів. Також встановлено, що INNER JOIN залишає лише рядки з парою в усіх таблицях, тоді як LEFT і RIGHT JOIN зберігають рядки без відповідника (наприклад, клієнтів без замовлень чи книги без відгуків), і в таких запитах слід використовувати COUNT(стовпець) замість COUNT(*), щоб не враховувати NULL. Підзапити дозволили порівнювати значення з обчисленими показниками, як-от середня ціна книги чи середня сума замовлення. Мету роботи досягнуто.